

NanoGPT Project Report

Idant Srivastava

June 2026

Contents

1	Data Pipeline	3
1.1	Lyrics Acquisition (<code>data/lyrics.py</code>)	3
1.2	Structured Annotation (<code>data/tagging.py</code>)	3
2	Corpus Analysis	3
2.1	Dataset Overview	3
2.2	Genre Distribution (Raps)	4
2.3	Mood Distribution (Raps)	4
2.4	Observations	4
3	Model Architecture	4
3.1	Tokenization	4
3.2	Embeddings and Positional Encoding	5
3.3	Self-Attention	5
3.4	Feed-Forward Network (FFN)	5
3.5	Residuals and LayerNorm	5
3.6	Full Transformer Block	5
3.7	Generation	5
4	Training	6
4.1	Training Setup	6
4.2	Models Trained	6
4.3	Final Loss Results	6
4.3.1	NanoGPT-1 (8 layers, 1 head, emb = 128)	6
4.3.2	NanoGPT-4 (4 layers, 4 heads, emb = 128)	6
4.3.3	NanoGPT-8 (8 layers, 8 heads, emb = 384)	6
4.3.4	NanoGPT-12 (8 layers, 12 heads, emb = 384)	6
4.4	Observations	7
5	Generation Comparison	7
5.1	Greedy Decoding (<code>top_k = 1</code>)	7
5.2	Temperature Sampling	7
5.3	Top- k Sampling	8
5.4	Top- p (Nucleus) Sampling	8
5.5	NanoGPT-8: Greedy Decoding (Repetition Failure)	8

5.6	Decoding Strategy Comparison	9
6	Failure Analysis	9
7	Final Report Questions	9
7.1	What is a Foundation Model?	9
7.2	Why Do Transformers Scale?	10
7.3	Why Do Models Hallucinate?	10
7.4	Why Does Data Matter More Than Expected?	10
7.5	Why Does Prompting Work?	10
7.6	Why Does the Same Model Behave Differently Across Runs?	11
7.7	What Are the Structural Limits of Decoder-Only Transformers?	11
7.8	What Would You Improve Next?	11
A	Repository Structure	13

1 Data Pipeline

1.1 Lyrics Acquisition (`data/lyrics.py`)

Lyrics were scraped from the Genius API using the `lyricsgenius` library. A curated list of 47 artists spanning several decades and sub-genres was assembled, covering classic boom-bap, contemporary trap, conscious rap, UK grime and drill, Punjabi rap, and experimental. The final collection was stored as a JSON array.

1.2 Structured Annotation (`data/tagging.py`)

Each song was annotated with four categorical tags using the **Gemma 4 26B MoE** model, enforcing structured JSON output through Pydantic schemas.

Attribute	Allowed Values
Mood	aggressive, confident, melancholic, introspective, motivational, hype, dark, romantic, vulnerable, rebellious, playful, emotional, storytelling
Genre	trap, boom_bap, drill, grime, conscious_rap, melodic_rap, experimental, underground, mainstream, alternative_hiphop
Rhyme Scheme	monorhyme, couplet, alternate_rhyme, internal_rhyme, multisyllabic, dense_internal, freeform
Cadence	slow, midtempo, fast, switching, staccato, rolling, bouncy

The final tagged dataset (`tagged_rap_dataset_gemma4.json`) contained **2 196 songs from 43 unique artists**.

2 Corpus Analysis

2.1 Dataset Overview

Two corpora were prepared in parallel and kept as separate binary files.

Split	Poems	Raps
Source files	20 657 .txt poems	2 196 JSON songs
Train tokens	~5.22M	~1.98M
Val tokens	~0.58M	~0.22M
Train/Val split	90 / 10%	90 / 10%

Poems were encoded directly (one poem per sequence, separated by `<|endoftext|>`). Rap sequences were prefixed with a conditioning header, for example:

```
[Genre:  conscious-rap] [Mood:  introspective] [Rhyme:  couplet] [Cadence:  midtempo] <lyrics text>
```

This conditioning scheme allows controllable generation at inference time.

2.2 Genre Distribution (Raps)

Genre	Count
conscious_rap	432
mainstream	417
trap	379
boom_bap	238
grime	179
other	551

2.3 Mood Distribution (Raps)

Mood	Count
confident	729
introspective	385
aggressive	325
melancholic	203
rebellious	151
other	403

2.4 Observations

- The corpus spans **43 distinct artists** with an average song length of approximately 3 133 characters (≈ 700 tokens after GPT-2 BPE encoding).
- The dominant cadence is **midtempo** (55%), reflecting the broad stylistic mainstream.
- The dominant rhyme scheme is **multisyllabic** (40%), consistent with the lyrical density of the selected artists (e.g., Kendrick Lamar, J. Cole, MF DOOM).
- The poems corpus is approximately $2.6\times$ larger by token count, making it suitable as a pre-training stage before fine-tuning on the more structured rap data. ‘

3 Model Architecture

All four model variants share the same **GPT-2-style decoder-only transformer** architecture implemented in `model/model.py`. The architecture is composed of the following components:

3.1 Tokenization

Text is converted into a sequence of integer IDs using the **GPT-2 Byte-Pair Encoding (BPE)** tokenizer (via the `tiktoken` library). This provides a vocabulary size of 50 257 tokens, effectively handling sub-word structures and unseen words.

3.2 Embeddings and Positional Encoding

Each token ID is mapped to a dense vector space using a **Token Embedding** matrix. Since transformers lack inherent notions of sequence order, a **Positional Encoding** is added to each token embedding. We use learned positional embeddings corresponding to a maximum context length of 256 tokens. The token embedding matrix is tied with the final LM head to reduce parameter count and improve generalisation.

3.3 Self-Attention

The core mechanism is **Causal Multi-Head Self-Attention**. It projects the input into Query, Key, and Value matrices. To preserve the autoregressive property, a causal mask is applied to prevent tokens from attending to future tokens. We utilise PyTorch’s `scaled_dot_product_attention` with `is_causal=True` for efficient computation.

3.4 Feed-Forward Network (FFN)

Following attention, the representations pass through a **Feed-Forward Network** (or MLP). It consists of two linear transformations with a GELU activation in between. The inner dimension is expanded by a factor of 4 relative to the embedding dimension, allowing the model to learn complex non-linear transformations.

3.5 Residuals and LayerNorm

We employ a **Pre-LayerNorm** configuration, applying Layer Normalization before the attention and FFN sub-layers. **Residual connections** bypass these sub-layers and add their output back to the main stream, aiding gradient flow during training. Additionally, residual projections are scaled by $\sigma = 0.02 / \sqrt{2L}$ at initialization, following the GPT-2 paper.

3.6 Full Transformer Block

A single **Transformer Block** chains these operations: the input is normalized, passed through self-attention, and added to the residual stream. It is then normalized again, passed through the FFN, and added to the residual stream. The models stack 4 or 8 of these blocks.

3.7 Generation

During inference, the model generates text autoregressively. Given a prompt, it predicts the next token’s probability distribution. Decoding supports **greedy**, **temperature sampling**, **top- k** , **top- p (nucleus)**, and **repetition penalty**.

4 Training

4.1 Training Setup

Setting	Value
Optimiser	AdamW
Batch size	32 sequences
Context length	256 tokens
Tokeniser	GPT-2 BPE (<code>tiktoken</code>), vocab 50 257
Stage 1	Pre-train on poems, $\text{lr} = 10^{-3}$
Stage 2	Fine-tune on tagged raps, $\text{lr} = 3 \times 10^{-4}$
Evaluation	Cross-entropy every 250 steps over 200 batches

4.2 Models Trained

Model	Layers	Heads	Embed dim	Params	Dropout
NanoGPT-1	8	1	128	$\sim 8.0\text{M}$	0.1
NanoGPT-4	4	4	128	$\sim 1.1\text{M}$	0.1
NanoGPT-8	8	8	384	$\sim 33.5\text{M}$	0.1
NanoGPT-12	8	12	384	$\sim 33.5\text{M}$	0.1

4.3 Final Loss Results

4.3.1 NanoGPT-1 (8 layers, 1 head, emb = 128)

Dataset	Iters	Train Loss	Val Loss	Val Perplexity
Poems	2000	4.7146	5.1184	167.1
Raps	2000	3.6071	4.4043	81.8

4.3.2 NanoGPT-4 (4 layers, 4 heads, emb = 128)

Dataset	Iters	Train Loss	Val Loss	Val Perplexity
Poems	2000	4.6548	5.0827	161.2
Raps	2000	3.6111	4.4663	87.1

4.3.3 NanoGPT-8 (8 layers, 8 heads, emb = 384)

Dataset	Iters	Train Loss	Val Loss	Val Perplexity
Poems	2000	4.6080	5.0492	156.3
Raps	2000	3.1539	4.3658	78.8

4.3.4 NanoGPT-12 (8 layers, 12 heads, emb = 384)

Dataset	Iters	Train Loss	Val Loss	Val Perplexity
Poems	1000	5.1193	5.4843	240.8
Raps	1500	3.5355	4.5051	90.5

4.4 Observations

- **NanoGPT-1** (single-head attention) achieves a validation loss on raps of 4.40. Despite having only one attention head, it performs close to the multi-head models and outperforms the shallower **NanoGPT-4**, demonstrating that increased depth (8 layers) and capacity can mitigate the lack of multiple heads.
- **NanoGPT-8** achieves the lowest validation loss on raps (4.37), suggesting the 8-head, 384-dim configuration is a sweet spot for this corpus size.
- **NanoGPT-4** trains fastest (~ 455 s for raps) but plateaus early, indicating an underfitting regime.
- **NanoGPT-12** was trained for fewer iterations due to Colab time constraints; its higher validation loss reflects this rather than inferior architecture.
- All models show a widening **train–val gap on raps** after step ~ 1000 , consistent with overfitting to the small (~ 2.2 M token) rap corpus.
- Fine-tuning benefits significantly from the lower learning rate (3×10^{-4}), allowing gradual adaptation of poem-pre-trained representations to rap structure.

5 Generation Comparison

All samples below are from the **NanoGPT-12** checkpoint unless stated. Prompt:

[Genre: melodic-rap] [Mood: vulnerable] [Rhyme: internal-rhyme] [Cadence: midtempo] Yeah, look, it's 3 AM in Toronto

5.1 Greedy Decoding (**top_k = 1**)

[Genre: conscious] [Mood: peaceful] [Rhyme: simple] [Cadence: slow] Look, I was born with a curse, but I'm a pacifist who hates violence? I'm the same one of my life is in this world And if you're just a little weapon and a man You can't be your soul, it's not a king But it's been a long time to be a long time So I don't know what we are It's gonna be a long time for us The only one that we've done changed We'll never see how much we have to do They say they woulda had to go

Analysis. Greedy decoding is fully deterministic but falls into repetitive, low-entropy loops. The model immediately diverges from the conditioning tags into generic phrases. This illustrates “mode collapse” behaviour: always picking the single most-likely token produces common, context-free output.

5.2 Temperature Sampling

Settings: temperature=0.7, repetition_penalty=1.2

[Genre: melodic-rap] [Mood: vulnerable] [Rhyme: internal-rhyme] [Cadence: midtempo] Yeah, look, it's 3 AM in Toronto You know that we got that I came

on my side Make a dollar, nigga – you said you were too much But I was a man since they spent his momma as well I'm not scared of the one, everybody done like him; you're only a son? They tell me "Huh?!", and your best friend" And you really scared to say "Hell a brother?" No one, no, he just play on some more, yeah!

Analysis. $T = 0.7$ softens the distribution without making it fully random. The model respects the conditioning tags much better, producing text that reads like a coherent verse. Minor grammatical errors persist but the emotional register (vulnerable, introspective) is correctly captured.

5.3 Top- k Sampling

Settings: $top_k=15$, $repetition_penalty=1.2$

[Genre: melodic-rap] [Mood: vulnerable] [Rhyme: internal-rhyme] [Cadence: midtempo] Yeah, look, it's 3 AM in Toronto You see your neck on my knees And you're not so big, and you can't come to me? (I swear I'm too high) A-up – I'm tryna keep a mama' (Straight up), but ya (Ha-ha, hey) But you never gonna give him out of me If this is how much you got my ass for a bitch for me (Uh, yeah, yeah) That'll never let that you ride

Analysis. Restricting to 15 candidates per step avoids rare token selection while maintaining diversity. The output is noticeably more idiomatic than greedy: the model correctly inserts parenthetical ad-libs (common in rap) and lines flow more naturally, though meaning is occasionally fragmented.

5.4 Top- p (Nucleus) Sampling

Settings: $top_p=0.8$, $repetition_penalty=1.2$

[Genre: melodic-rap] [Mood: vulnerable] [Rhyme: internal-rhyme] [Cadence: midtempo] Yeah, look, it's 3 AM in Toronto You never need to die too long as a mill' that you know when we're gone

Analysis. Top- $p = 0.8$ dynamically adjusts the candidate set per step, allowing the model to terminate early once the EOS token enters the top 80% of cumulative probability mass. The short output reflects the model's confidence that the sequence is complete — a useful property for producing concise completions without forced truncation.

5.5 NanoGPT-8: Greedy Decoding (Repetition Failure)

Look, his palms are sweaty, knees weak, arms are heavy And I'm a savage, my diamonds is falling, hands is falling (Yeah) I'm a savage, I'm a savage, I'm a savage, I'm a savage, ... [repeated 80+ times]

Analysis. Without a repetition penalty, the 8-layer model's greedy path degenerates into an infinite loop. The model assigns the highest probability to the most recently seen phrase, creating a feedback cycle. This is a well-known failure mode: the training

objective does not penalise self-repetition directly.

5.6 Decoding Strategy Comparison

Strategy	Deterministic	Diverse	Coherent	Best Use
Greedy	Yes	Low	Low	Debugging, shortest path
Temperature	No	Medium	Medium	Creative generation
Top- k	No	High	Medium	Balanced quality
Top- p	No	Adaptive	High	Production / variable-length

6 Failure Analysis

During evaluation, several distinct failure modes were observed. We can classify these failures based on their root causes across the modelling pipeline:

- **Hallucinations:** The model generates fluent but factually incorrect or semantically nonsensical text (e.g., hallucinating non-existent artists or contradicting the prompt). *Cause: Data issue (noise in training data) & Inference issue (high temperature sampling dispersing probability mass).*
- **Repetition loops:** The model gets stuck repeating the same phrase (e.g., “I’m a savage, I’m a savage...”). *Cause: Inference issue (lack of repetition penalty during greedy decoding) & Architecture issue (standard causal objective doesn’t penalise repetition).*
- **Context forgetting:** The model ignores early conditioning tags (e.g., [Mood: vulnerable]) and drifts into generic patterns. *Cause: Architecture issue (fixed context window and causal masking weakening early token influence) & Training issue (insufficient conditioning enforcement).*
- **Contradictions:** The model does not contradict itself within the same verse (e.g., claiming to be rich, then immediately claiming to be poor) It may contradict itself sometimes depending on the context length. *Cause: Architecture issue (lack of explicit planning or external memory) & Inference issue (myopic next-token prediction).*

7 Final Report Questions

7.1 What is a Foundation Model?

A **foundation model** is a large neural network trained on broad, general-purpose data at scale, typically via self-supervised objectives such as next-token prediction or masked language modelling and then adapted (fine-tuned, prompted, or distilled) to a wide range of downstream tasks.

7.2 Why Do Transformers Scale?

Transformers exhibit predictable, power-law scaling in loss with respect to compute, data, and parameter count. Several mechanisms underpin this:

- **Parallel computation** — unlike RNNs, attention is computed over all token pairs simultaneously, enabling full GPU/TPU utilisation at any depth.
- **Universal approximation with depth** — each additional layer learns increasingly abstract representations; residual connections ensure stable gradient flow.
- **Weight sharing via attention** — the same projection matrices apply to every position, allowing the model to generalise patterns across contexts.

7.3 Why Do Models Hallucinate?

Hallucination arises from the mismatch between the training objective and factual accuracy. During training, the model maximises the likelihood of the *next token given the context*, not the likelihood of a *true fact*. Several factors cause this:

- **Distributional gaps** — if a fact is rare in training, the model interpolates between related patterns rather than recalling the exact answer.
- **Confidence without grounding** — the softmax always sums to 1; the model must assign probability *somewhere*, even for unknown facts.
- **Conditioning drift** — as generation proceeds, early conditioning tokens lose influence relative to the recently generated context.

7.4 Why Does Data Matter More Than Expected?

The dominant intuition in early deep learning was that architecture and optimisation were the primary levers. Scaling research has repeatedly shown that *data quantity and quality* are equally important:

- **Coverage** — a model can only generalise to concepts represented in training data. Our rap model struggles with under-represented cadences (staccato, rolling, <5% of corpus) due to too few examples.
- **Token economics** — neural scaling laws show that for a fixed compute budget, training a smaller model on more data often outperforms a larger model on less.
- **Diversity** — our corpus overly samples confident and introspective moods (55% combined), which explains why unconditioned generation lean toward those samples.

7.5 Why Does Prompting Work?

Prompting leverages the model’s in-context learning ability: the transformer’s attention mechanism allows the model to use the prompt as a soft demonstration of the desired output format, style, and content — without any weight update.

Attention heads at inference time perform an implicit Bayesian update: the prompt constrains the posterior over next tokens by providing evidence that shifts probability mass toward the desired subspace.

In our experiments, prepending `[Genre: boom_bap]` `[Mood: aggressive]` reliably produced harder, rhythmically denser output than an unconditioned prompt, because the conditioning header at the start of each training sequence taught the model to treat those tokens as style anchors.

7.6 Why Does the Same Model Behave Differently Across Runs?

1. **Sampling** — `torch.multinomial` draws from the categorical distribution at each step; different draws shift all subsequent probabilities.
2. **Random seed** — without `torch.manual_seed`, the state differs across runs. Evaluation notebooks fixed `torch.manual_seed(42)` for reproducible comparisons.
3. **Training run variation** — different training seeds produce checkpoints with measurably different loss curves, especially for small corpora where mini-batch ordering has a large effect.

7.7 What Are the Structural Limits of Decoder-Only Transformers?

- **Fixed context window** — sequences longer than `block_size=256` tokens are truncated; long-range structure (verse–chorus–bridge) cannot be modelled.
- **Left-to-right only** — causal masking makes the model unsuited to tasks requiring bidirectional context.
- **Quadratic attention complexity** — self-attention scales as $O(T^2)$ in sequence length, making long-context generation expensive..
- **No explicit planning** — the model cannot plan a rhyme scheme before writing the first word of a couplet.

7.8 What Would You Improve Next?

1. **Scale the corpus** — 2 196 songs (~ 2 M tokens) is well below the threshold for stable generalisation; targeting $10\times$ more songs would close the train–val gap.
2. **Longer context window** — increasing `block_size` to 512 or 1 024 would allow the model to attend across full verses and maintain structural coherence.
3. **Better conditioning** — replacing free-text tags with learned style embeddings would improve controllability.
4. **Evaluation beyond perplexity** — implement rhyme density metrics, and human evaluation of coherence.
5. **Instruction fine-tuning** — add an SFT stage where the model follows natural language instructions (e.g., “write a Kendrick-style introspective verse about identity”).

Conclusion

This project demonstrated the complete lifecycle of a language model: from web-scraped raw text, through structured annotation with a state-of-the-art LLM, to tokenisation, training, and multi-strategy evaluation. Four model variants were trained across two domains, with the 33.5M-parameter, 8-layer NanoGPT achieving the best validation loss on raps (4.37). The generation comparison clearly illustrated the trade-offs between determinism, diversity, and coherence across decoding strategies. The reflective questions surface the deeper principles governing why these systems work, where they fail, and what drives the field forward.

A Repository Structure

NanoGPT Project Structure

```
NanoGPT/
|-- data/
|   |-- lyrics.py           # Genius API scraper
|   |-- tagging.py         # Gemma-4 annotation
|       pipeline
|   |-- poems/             # 20,657 poem .txt files
|   |-- raps/
|       |-- cleaned_combined.json # 2,196 tagged rap songs
|
|-- tokenization/
|   |-- prepare_datasets.py # BPE tokenizer
|   |-- poems_train.bin     #bin files
|   |-- poems_val.bin
|   |-- raps_train.bin
|   |-- raps_val.bin
|
|-- model/
|   |-- model.py            # GPT, MLP, Block,
|                           # CausalSelfAttention
|
|-- training/
|   |-- training_pipeline_1.ipynb
|   |-- training_pipeline_4.ipynb
|   |-- training_pipeline_8.ipynb
|   |-- training_pipeline_12.ipynb
|   |-- training_pipeline_ffn_reduced.ipynb
|   |-- nanogpt_checkpoint_{1,4,8,12,ffn_reduced}.pt
|
|-- evaluation/
|   |-- eval_1.ipynb
|   |-- eval_4.ipynb
|   |-- eval_8.ipynb
|   |-- eval_ffn_reduced.ipynb
|   |-- eval_12.ipynb
```